[1]. Tool: ChatGPT
Prompt: Write Python code to extract only the animal data from the CIFAR-10 real data set, given the following instructions from the site and a similar loading method from my previous project.

Output:
Got you — here is the clean, correct version for loading colored CIFAR-10 animal images into X and y.

This will:

Load bird, cat, deer, dog, frog, horse

Keep them in color (RGB, 32×32)

Assign labels 0–5

Produce numpy arrays ready for training

✅ Colored CIFAR-10 Animals Loader (Final Code)

```
import os
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

# Your animal folders
birdpath  = "/code/COE379L-Final/cifar10_animals/bird"
catpath   = "/code/COE379L-Final/cifar10_animals/cat"
deerpath  = "/code/COE379L-Final/cifar10_animals/deer"
dogpath   = "/code/COE379L-Final/cifar10_animals/dog"
frogpath  = "/code/COE379L-Final/cifar10_animals/frog"
horsepath = "/code/COE379L-Final/cifar10_animals/horse"

# Map classes to labels
paths = {
    "bird": birdpath,
    "cat": catpath,
    "deer": deerpath,
    "dog": dogpath,
    "frog": frogpath,
```

```
    "horse": horsepath
}

label_map = {name: i for i, name in enumerate(paths.keys())}
print("Label mapping:", label_map)

X = []
y = []

# Load colored images
for name, path in paths.items():
    for img_name in os.listdir(path):
        if img_name.lower().endswith((".png", ".jpg", ".jpeg")):
            img_path = os.path.join(path, img_name)

            # Load as RGB, NOT grayscale
            img = load_img(img_path, color_mode="rgb", target_size=(32, 32))

            img_array = img_to_array(img)     # shape: (32, 32, 3)
            X.append(img_array)
            y.append(label_map[name])

# Convert to numpy arrays
X = np.array(X)
y = np.array(y)

print("Loaded X shape:", X.shape)   # → (num_images, 32, 32, 3)
print("Loaded y shape:", y.shape)   # → (num_images,)
```

[2] Tool: ChatGPT
Prompt: does this achieve what i think it should dat[dat['Stage'] == 'test']['Encoded_Class'] = dat[dat['Stage'] == 'test']['Encoded_Class'].apply(lambda x: x - 2)
Output:

```
dat.loc[dat['Stage'] == 'test', 'Encoded_Class'] = (
    dat.loc[dat['Stage'] == 'test', 'Encoded_Class'] - 2
)
```

[3] Tool: Google AI
Prompt: How to shuffle pandas df

Output:
```
 import pandas as pd

# Example DataFrame
data = {'A': range(1, 6), 'B': ['a', 'b', 'c', 'd', 'e']}
df = pd.DataFrame(data)

# Shuffle the DataFrame
shuffled_df = df.sample(frac=1)

# Optional: Reset the index to have a clean, sequential 0-based index
shuffled_df = shuffled_df.reset_index(drop=True)

print(shuffled_df)
```

[4] Tool: ChatGPT
Prompt: finish this generate/ route from flask import Flask, Response, request import numpy as np import matplotlib.pyplot as plt import matplotlib.image as mpimg from skimage.color import rgb2gray import tensorflow as tf import io from PIL import Image import transformers from transformers import pipeline from diffusers import DiffusionPipeline from diffusers import AutoPipelineForText2Image app = Flask(__name__) model10 = tf.keras.models.load_model('models/model10.keras') real_model = tf.keras.models.load_model('models/lenet5_model_realCIFAR.keras') pipe = AutoPipelineForText2Image.from_pretrained("stabilityai/sd-turbo", variant="fp16") def create_image(prompt): im = pipe(prompt=prompt, num_inference_steps=1, num_images_per_prompt=1, guidance_scale=0.0).images[0] return im @app.route('/generate', methods=['POST']) def @app.route('/models/<model_type>', methods=['GET']) def model_info(model_type): if model_type == "real": return { "model" : "Lenet_5 arhcitecture trained on 100% real data", "version": "v1", "name": "animal_classifier", "description": "Classify dog, cat, bird, frog, horse, and deer images.", "number_of_parameters": 2601090 } elif model_type == "mixed": return { "model" : "Lenet_5 arhcitecture trained on 46% real data", "version": "v1", "name": "animal_classifier", "description": "Classify dog, cat, bird, frog, horse, and deer images.", "number_of_parameters": 2601090 } def preprocess_image(im): im = im.resize((32,32), Image.LANCZOS) arr = np.array(im) arr = arr/255.0 arr = np.expand_dims(arr, axis=-1) arr = np.expand_dims(arr, axis=0) return arr @app.route('/inference/<model>', methods=['POST']) def classify(model): # check if the post request has the file part if 'image' not in request.files: return {"error": "Invalid request; pass a binary image file as a multi-part form under the image key."} # get the data im = request.files['image'] if not im: return {"error": "The `image` field is required"}, 404 try:

image_stream = io.BytesIO(im.read()) im = Image.open(image_stream) except Exception as e: return {"error": f"Could not read file bytes details: {e}"}, 404 try: data = preprocess_image(im) except Exception as e: return {"error": f"Could not process the `image` field; details: {e}"}, 404 if model == "mixed": result = model10.predict(data) elif model == "real": result = real_model.predict(data) prediction = np.argmax(result, axis = 1) if prediction == 0: classification = 'bird' elif prediction == 1: classification = 'cat' elif prediction == 2: classification = 'deer' elif prediction == 3: classification = 'dog' elif prediction == 4: classification = 'frog' elif prediction == 5: classification = 'horse' else: return {"error" : "No classification"} return {"prediction": classification} # start the development server if __name__ == '__main__': app.run(debug=True, host='0.0.0.0')
Output:

```
@app.route('/generate', methods=['POST'])
def generate_image():
    # Expect JSON request
    data = request.get_json()

    if not data or "prompt" not in data:
        return {"error": "Missing JSON field `prompt`"}, 400

    prompt = data["prompt"]

    try:
        # Generate image using your diffusion pipeline
        im = create_image(prompt)

        # Convert PIL.Image → JPEG bytes
        img_io = io.BytesIO()
        im.save(img_io, format='JPEG')
        img_io.seek(0)

        # Return the image
        return Response(img_io.getvalue(), mimetype='image/jpeg')

    except Exception as e:
        return {"error": f"Image generation failed: {e}"}, 500
```

```
curl -X POST http://localhost:5000/generate \
    -H "Content-Type: application/json" \
    -d '{"prompt":"a cute orange cat"}' \
```

--output output.png